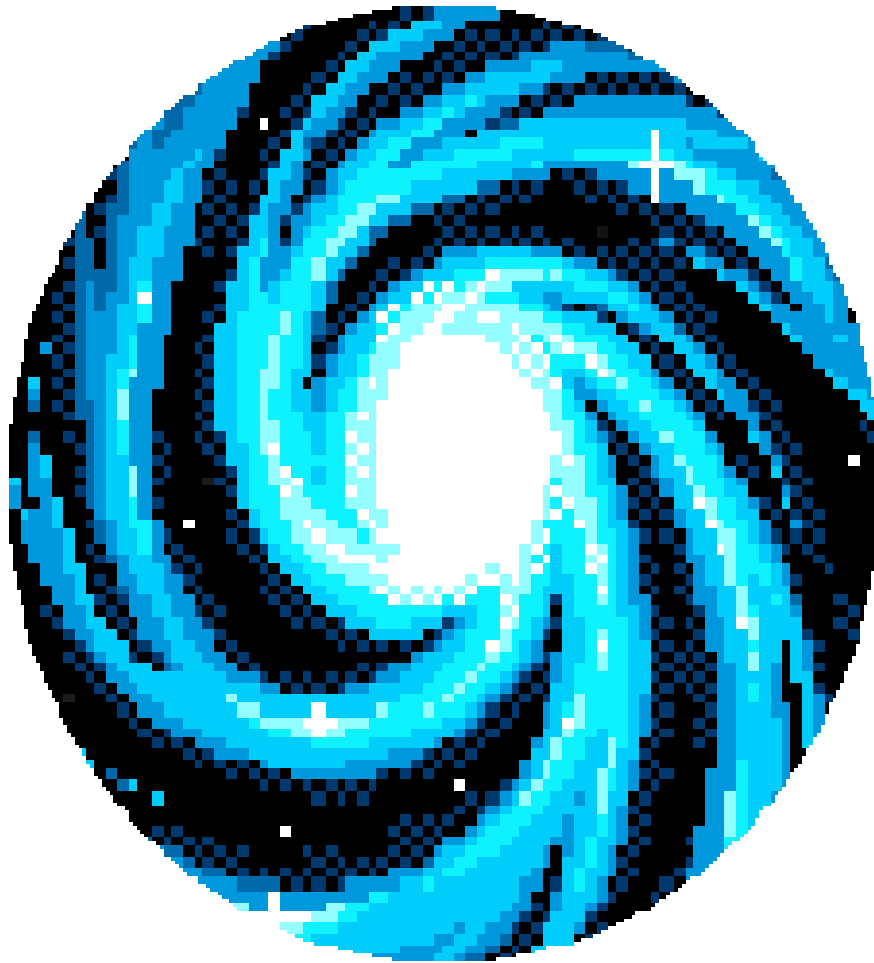


Escape From Valusia

Software Requirements Specification



1. Introduction

Purpose

This document outlines the software architecture for the game *Escape From Valusia*. It is intended to provide documentation for future project collaborators and investors.

Scope

Escape From Valusia, codenamed **Project Reptile**, is a dungeon-crawling puzzle game inspired by classic Windows titles like *Minesweeper* and *Quenzar's Caverns*. Designed for quick gameplay sessions (under one hour), it will be deployed as a C# WinForms desktop application, with a future mobile version in planning.

Overview

This document includes the following sections:

- **Section 2:** Overall Description
- **Section 3:** Game Mechanics
- **Section 4:** User Interface
- **Section 5:** Art Direction
- **Section 6:** Class Hierarchy and Software Architecture

2. Overall Description

Product Perspective

Escape From Valusia is set in a mysterious, fantasy world heavily inspired by the writings of H.P. Lovecraft. Many of Lovecraft's public domain works form the thematic backbone of the game's setting, characters, and atmosphere.

The game's mechanics draw inspiration from *Quenzar's Caverns*, blending classic dungeon crawler exploration with the trap-detection logic of *Minesweeper*. This unique combination is currently not found in desktop or mobile titles, offering an original gameplay experience.

The initial release targets Windows desktop systems, with plans to expand to mobile platforms after launch.

3. Game Mechanics

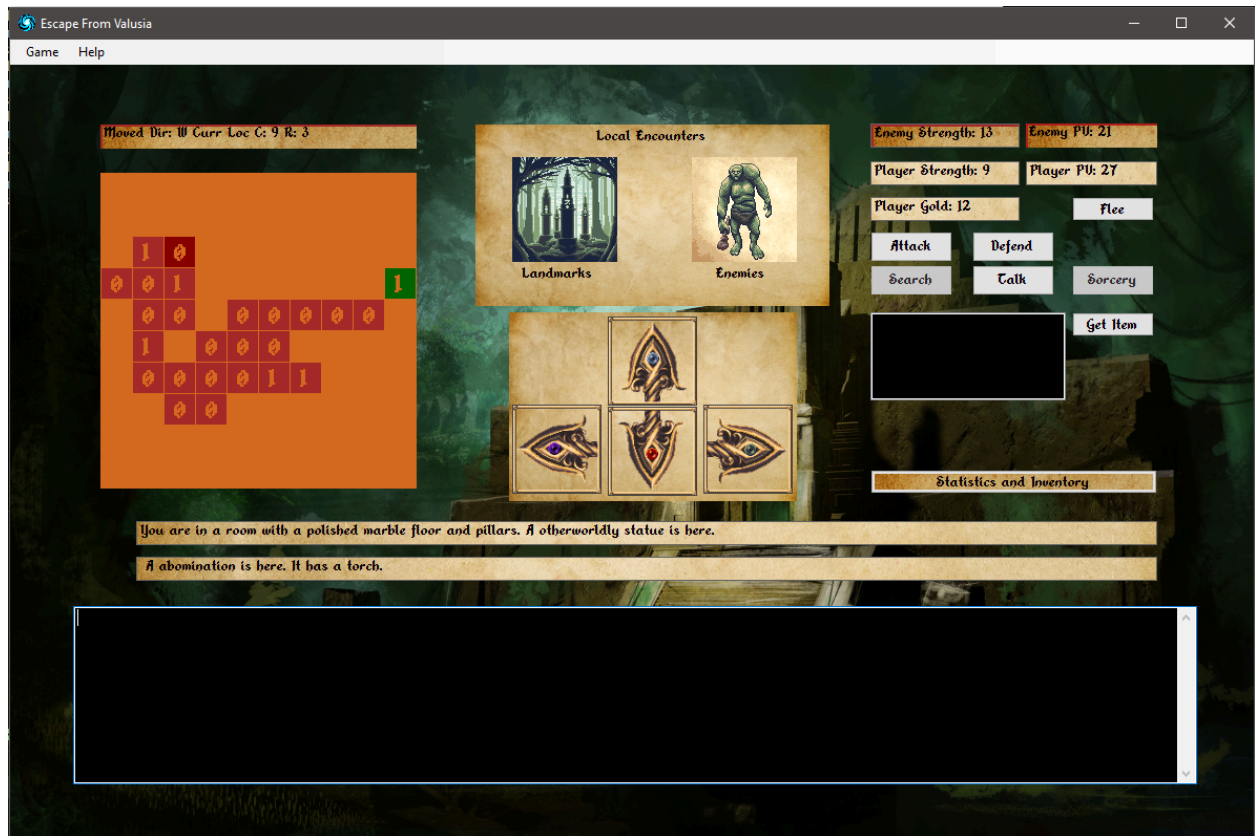
Project Reptile is a grid-based dungeon crawler with puzzle mechanics. Core gameplay features include:

- **10x10 Randomized Dungeon Grid:**
Each tile may contain enemies, landmarks, traps—or a combination thereof. When a player steps on a tile, a number indicates how many traps are adjacent, echoing *Minesweeper*'s mechanics.
- **Dynamic Enemy System:**
Over 20 unique enemies inspired by Lovecraft's mythos, each with original pixel art. Enemy and boss encounters are randomized each game to ensure replayability.
- **Item Collection & Customization:**
More than 30 unique items, including weapons, armor, consumables, and magical artifacts. Players must strategize loadouts to stand a chance against the final boss.

4. User Interface

- **Main Interface:**

Most game play takes place from a centralized UI, which displays a map showing the players location, buttons for action and movement, information regarding the room you are in, and a display of your main stats as well as those of any enemies in the room.



On the left-hand side of the GUI, a **10x10 grid** is displayed, representing the dungeon map. This grid consists of **100 individual tiles**, each visually indicating its current state:

- The **player's location** is highlighted in **green**.
- **Traversed tiles** (those the player has already stepped on) appear **brown**.
- **Trap tiles**, when revealed, are displayed in **red**.
- Tiles may also display a **number**, indicating how many traps are located in the adjacent tiles (up to 8 surrounding tiles). This mechanic helps the player make informed decisions and avoid dangerous areas, similar to *Minesweeper*

The top center of the screen displays any **Enemy** or **Landmark** encounter icons.

On the right-hand side of the GUI, a group of labels show the player's **Strength** (HP) and **PV** (power value, an aggregate of player stats), as well as the player's carried amount of gold. Also shown are the same stats of any enemies that share a location with the player, if any.

Below this are action buttons for the player.

- The **Attack** button, which will allow the player to attempt an attack upon the enemy.
- The **Defend** button, which will make an attack attempt against the enemy with reduced dexterity but an increased block chance.
- The **Sorcery** button, which will cast a sorcery if a tome is equipped.
- The **Search** button, which will search a landmark at the player's location.
- The **Talk** button, which will initiate negotiation with intelligent enemies.
- The **Flee** button, which will make a flee attempt to escape engagement with an enemy.
- The **Get Item** button, which will add the item selected in the adjacent ListBox to the player's inventory if the player has room in their inventory.
- The **Statistics and Inventory** button which launches the Statistics and Inventory subform.

In the direct center of the GUI, are **Movement** buttons that allow the player to move **North**, **South**, **East** or **West**. In this game, the map grid wraps around, meaning a player moving through the east side of the grid will end up on the first tile in the column on the westmost side of the tile grid.

The lower portion of the GUI has a pair of labels to describe the location the player is in, stored as unique strings within each parcel, one for the general parcel description and any landmarks if present, and one for information regarding enemies at the player's location. Below that is the **Player Console** to print game information to the player.

Statistics & Inventory Menu

This menu allows the player to manage their equipment and consumables, view current stats, and check their progress toward unlocking the **Inner Sanctum**.

Menu Features:

- **Player Stats Display:**

- View current statistics (e.g., health, strength, defense).
- See whether the **Key to the Inner Sanctum** has been found.

- **Inventory Management:**

- Browse through collected items.
- See which items are currently equipped.

Buttons & Functions:

- **Done:**

- Closes the Statistics & Inventory Menu and returns to the main game screen.

- **Use Item:**

- If the selected item is **consumable**: Uses the item and removes it from inventory.
- If the selected item is **equippable**: Attempts to equip the item, replacing any existing item in that slot.

- **Unuse Item:**

- Unequips the selected equippable item if it is currently in use.

- **Drop Item:**

- Removes the selected item from the inventory permanently.

Character Name:

Max Strength

Current St

Max Dexterity

Modified Dex

Intelligence:

Modified Int

Collected Gold:

Gold Key needed.

Ready Weapon: machete

Weapon

Ready Shield: small shield

Armour

Armour Worn: leather armor

machete
small shield
leather armor
Red Potion
Pink Potion

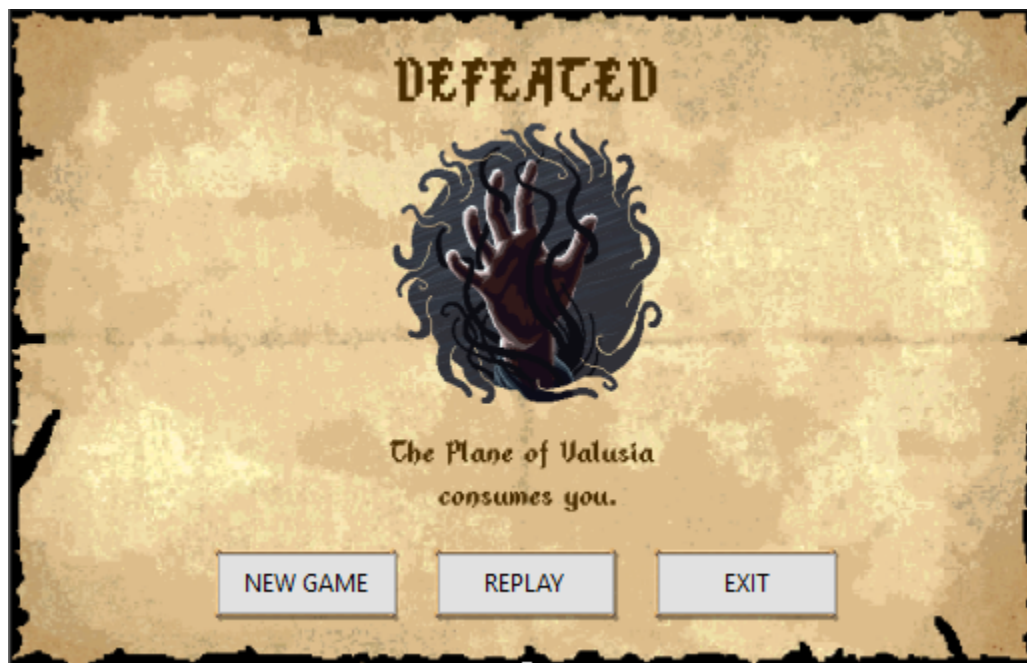
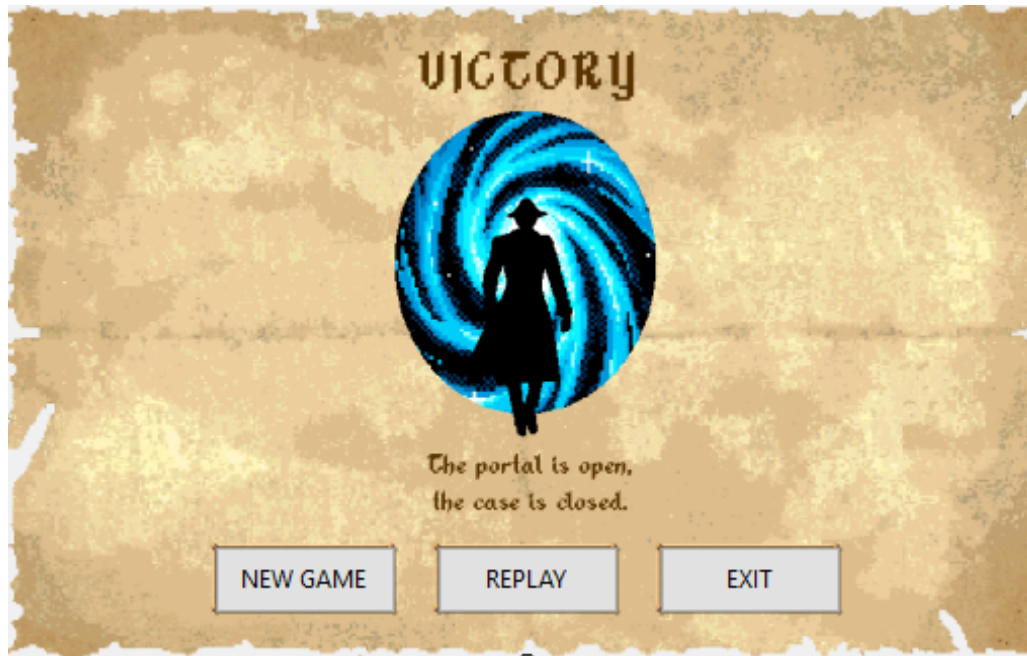
Done

Use Item

Unuse Item

Drop Item

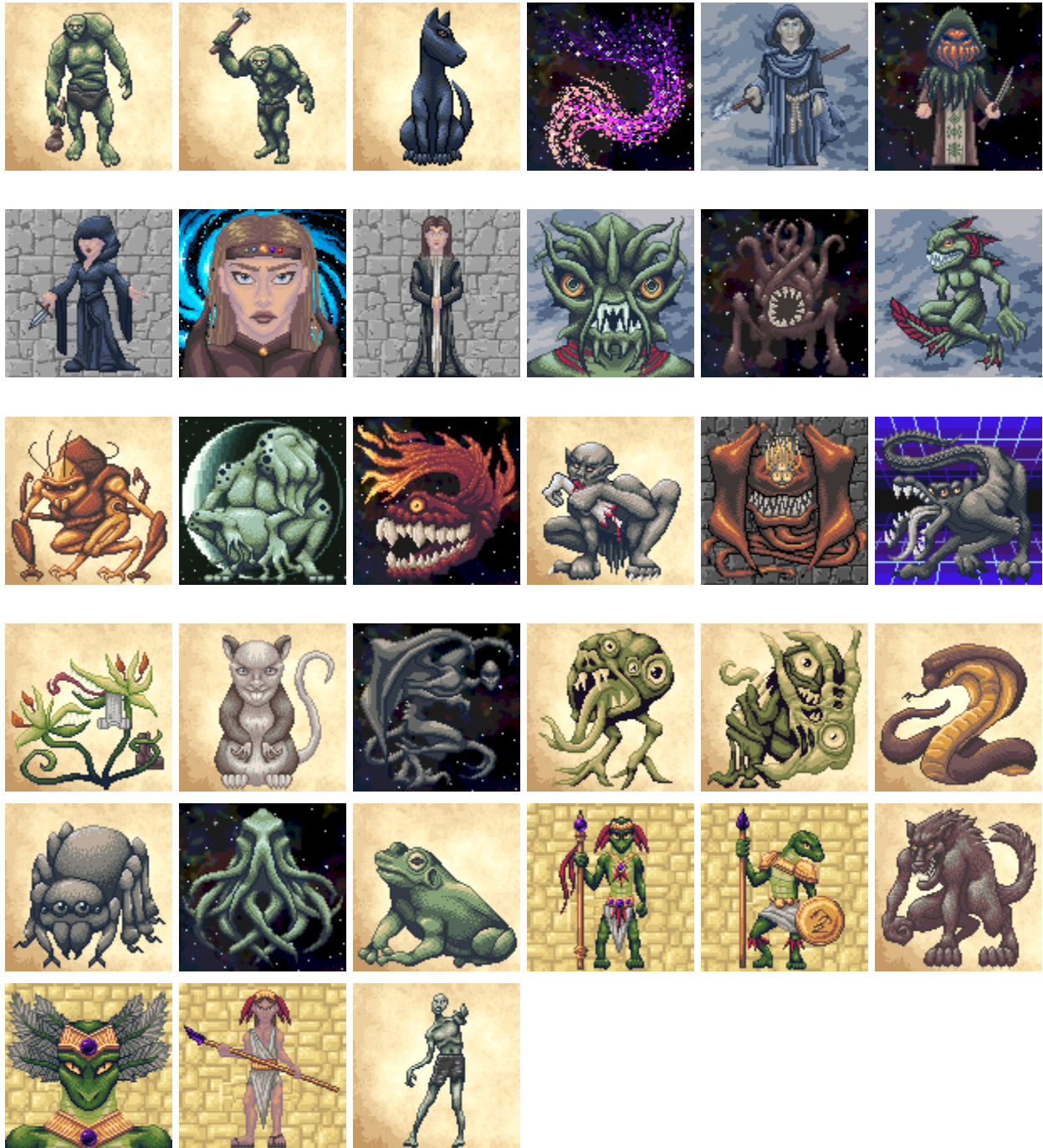
- **Endgame Screens (Victory & Death):**
Upon death or completion, players are presented with options to start a new game, replay the current game with all items and encounters reset, or exit the application.



5. Art Direction

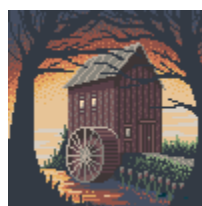
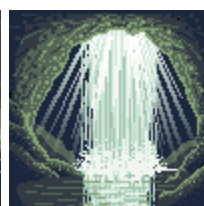
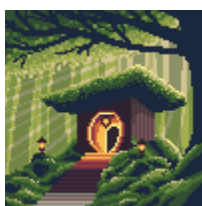
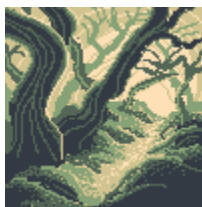
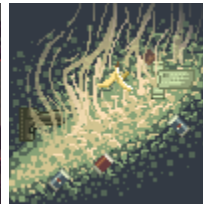
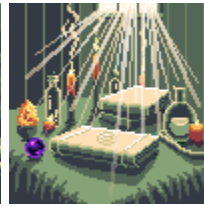
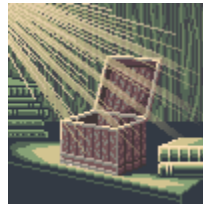
We are collaborating with Olympia-based artist **Rachel Webster** for original art assets, including enemies and landmarks. The visual style is influenced by 16-bit era games—vibrant, pixelated, and atmospheric, aiming for a “spooky” rather than overtly horrific aesthetic.

Monster Art



Landmark Art

Landmark Art:



6. Class Hierarchy and Software Architecture

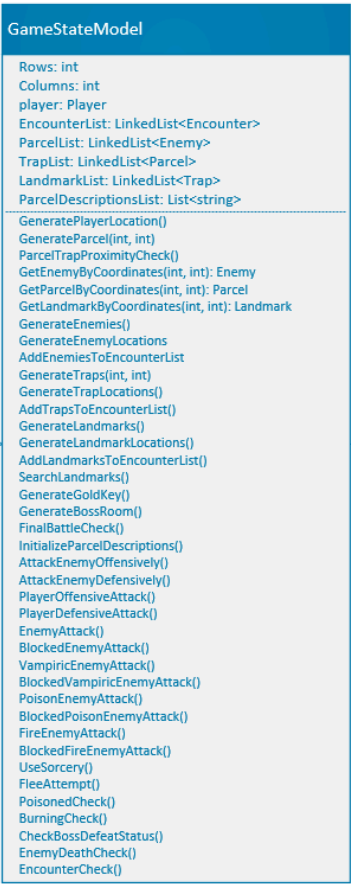
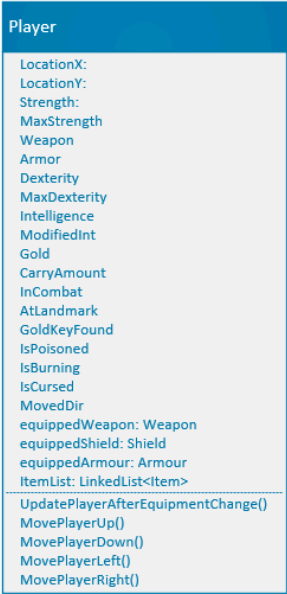
The game is being developed in **Windows Forms (WinForms)**, with an emphasis on decoupling game logic from UI components to facilitate seamless game resets and maintainable code architecture.

Core Gameplay Loop

Gameplay revolves around exploring the haunted forest of Valusia. Players collect items, battle enemies, and survive traps while trying to reach the Inner Sanctum.

Initialization Sequence:

1. Instantiate `GameStateModel` in `MainForm`
2. Randomly generate and assign:
 - Parcel descriptions (25 unique strings)
 - Enemy objects (via `EnemyFactory`)
 - 10 trap objects
 - 0–50 landmarks
 - 2 landmarks with unique items
3. Place enemies, traps, and landmarks on the map
4. Generate player starting location
5. Generate gold key and Inner Sanctum location
6. Initialize player object

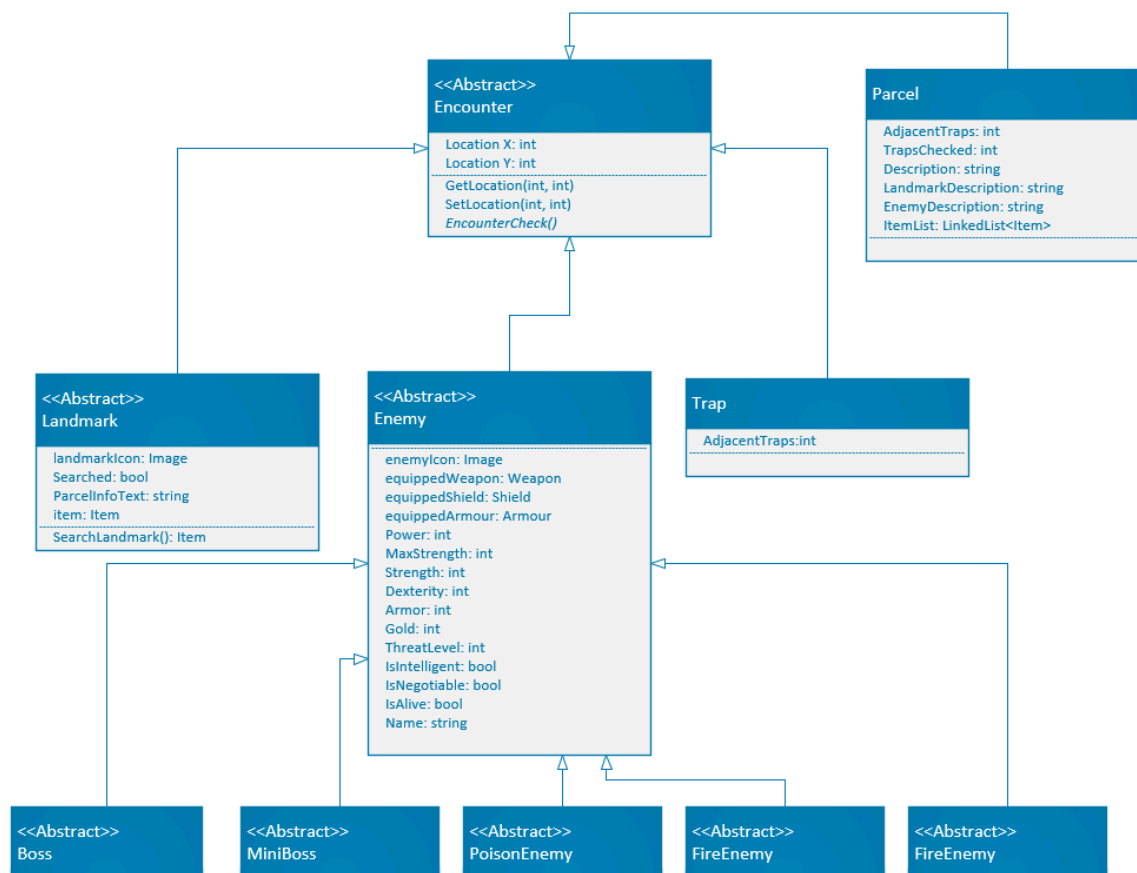


Encounter Class Hierarchy

The **Encounter** class is the base abstract class for all in-game interactions. It has the following hierarchy:

- **Trap** → Inherits **Encounter**
- **Parcel** → Inherits **Encounter**
- **Enemy** → Subclassed for type-specific logic and identity using **is** operator
- **Landmark** → Subclassed to represent diverse terrain or lore sites

Note: **Enemy** subclasses contain no additional logic but are useful for clean type-checking.



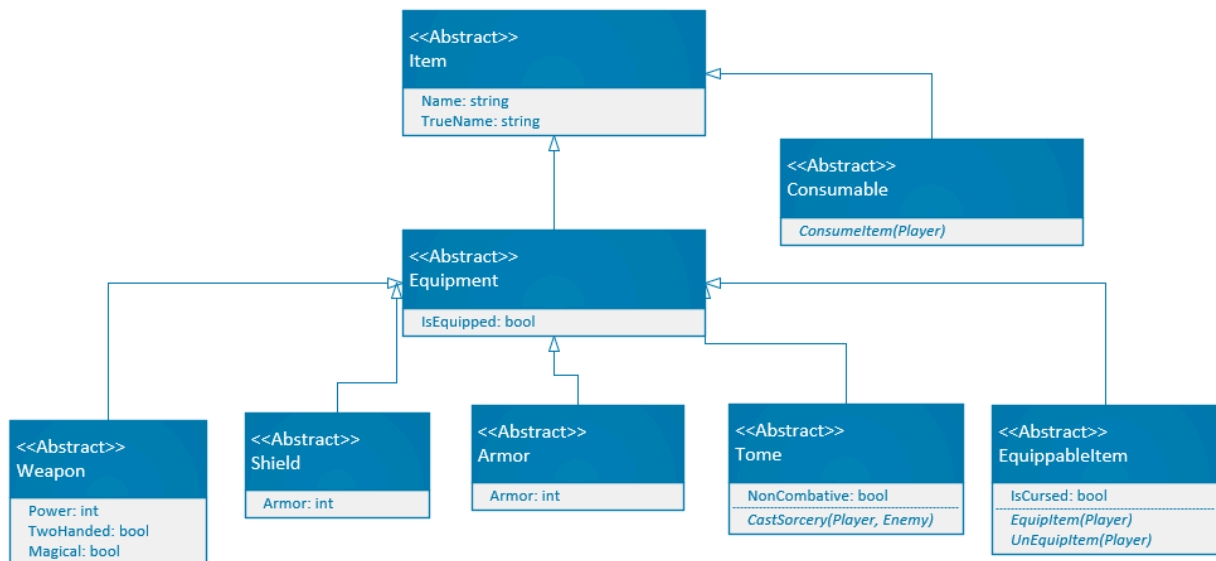
Item Class Hierarchy

Item types follow a dual-branch hierarchy:

- **Consumables:** One-time-use items that are removed after activation.
- **Equippables:** Provide persistent buffs or effects; may be cursed and require cleansing before unequipping.

Subtypes of Equippables:

- **Weapons:** One- or two-handed; two-handed weapons prevent shield use
- **Shields/Armor:** Increase defensive value
- **Tomes:** Enable spellcasting, which may backfire and harm the player



Object Factories

All randomized instantiations are handled through dedicated **Factory** classes, adhering to the **Simple Factory design pattern**, ensuring modular, testable code.

```
namespace ProjectReptile.Factories
{
    7 references
    class ShieldFactory
    {
        7 references
        public static Shield CreateShield()
        {
            Random random = new Random();

            int shield = random.Next(1, 5);

            switch (shield)
            {
                case 1:
                    return new SmallShield();

                case 2:
                    return new StandardShield();

                case 3:
                    return new GreatShield();

                case 4:
                    return new DragonShield();

            }

            return null;
        }
    }
}
```

<<Static>>
LandmarkFactory

[CreateLandmark\(\): Landmark](#)

<<Static>>
EnemyFactory

[CreateEnemy\(\): Enemy](#)
[CreateMiniBoss\(\): Enemy](#)

<<Static>>
ItemFactory

[CreateConsumeable\(\): Item](#)
[CreatePotion\(\): Item](#)
[CreateEquippableItem\(\): Item](#)
[CreateScroll\(\): Item](#)
[CreateTome\(\): Item](#)

<<Static>>
WeaponFactory

[CreateWeaponForCultist\(\): Weapon](#)
[CreateWeapon\(\): Weapon](#)

<<Static>>
ShieldFactory

[CreateShield\(\): Shield](#)

<<Static>>
ArmourFactory

[CreateArmor\(\): Armour](#)